

Chapter 17. Services

In [“Using a Pod’s IP Address for Network Communication”](#), we learned that you can communicate with a Pod by its IP address. A restart of a Pod will automatically assign a new virtual ClusterIP address. Therefore, other parts of your system cannot rely on the Pod’s IP address if they need to talk to one another.

Building a microservices architecture, where each of the components runs in its own Pod with the need to communicate with each other through a stable network interface, requires a different primitive, the *Service*.

The Service implements an abstraction layer on top of Pods, assigning a fixed virtual IP fronting all the Pods with matching labels, and that virtual IP is called ClusterIP. This chapter will focus on the ins and outs of Services, and most importantly the exposure of Pods inside and outside of the cluster based on their declared type.

ACCESSING A SERVICE IN MINIKUBE

Accessing Services of type `NodePort` and `LoadBalancer` in minikube requires special handling. Refer to the [documentation](#) for detailed instructions.

COVERAGE OF CURRICULUM OBJECTIVES

This chapter addresses the following curriculum objectives:

- Understand connectivity between Pods
 - Use ClusterIP, NodePort, LoadBalancer service types and endpoints
 - Understand and use CoreDNS
-

Working with Services

In a nutshell, Services provide discoverable names and load balancing to a set of Pods. The Service remains agnostic from IP addresses with the help of the Kubernetes DNS control-plane component, an aspect we’ll discuss in [“Discovering the Service by DNS lookup”](#). Similar to a Deployment, the Service determines the Pods it works on with the help of label selection.

[Figure 17-1](#) illustrates the functionality. Pod 1 receives traffic as its assigned label matches with the label selection defined in the Service. Pod 2 does not receive traffic, as it defines a nonmatching label.

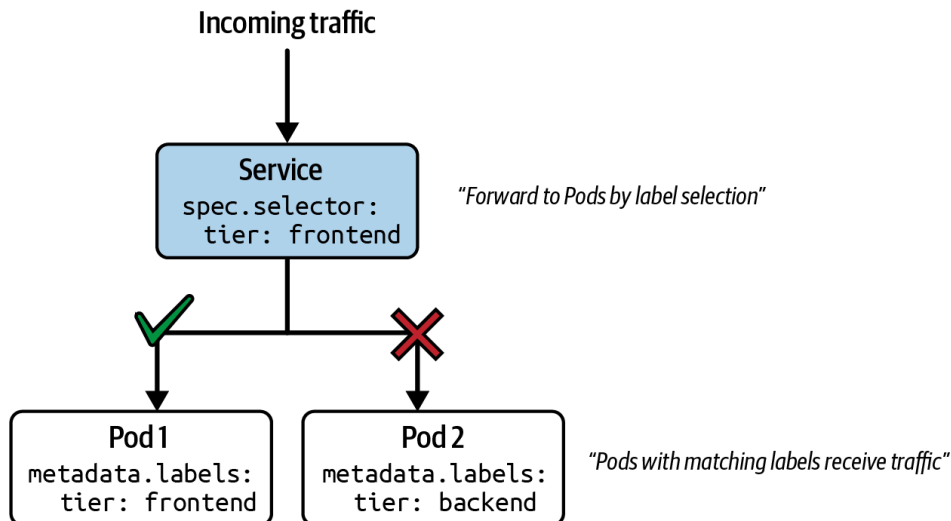


Figure 17-1. Service traffic routing based on label selection

Note that it is possible to create a Service without a label selector to support other scenarios. Refer to the relevant [Kubernetes documentation](#) for more information.

SERVICES AND DEPLOYMENTS

Services are a complementary concept to Deployments. Services route network traffic to a set of Pods, and Deployments delegate to a ReplicaSet to manage a set of Pods, the replicas. While you can use both concepts in isolation, it is recommended to use Deployments and Services together. The primary reason is the ability to scale the number of replicas and at the same time be able to expose an endpoint to funnel network traffic to those Pods.

Service Types

Every Service defines a type. The type is responsible for exposing the Service inside and/or outside of the cluster. [Table 17-1](#) lists the Service types relevant to the exam.

Table 17-1. Service types

Type	Description
<u>ClusterIP</u>	Exposes the Service on a cluster-internal IP. Reachable only from within the cluster. Kubernetes uses a round-robin algorithm to distribute traffic evenly among the targeted Pods.
<u>NodePort</u>	Exposes the Service on each node's IP address at a static port. Accessible from outside of the cluster. The Service type does not provide any load balancing across multiple nodes.
<u>LoadBalancer</u>	Exposes the Service externally using a cloud provider's load balancer.

Other Service types, e.g., `ExternalName` or the headless Service, can be defined; however, we'll not address them in this book, as they are not within the scope of the exam. For more information, refer to the [Kubernetes documentation](#).

Service type inheritance

The Service types just mentioned, `ClusterIP`, `NodePort`, and `LoadBalancer`, make a Service accessible with different scopes of exposure. It's imperative to understand that those Service types also build on top of each other. [Figure 17-2](#) shows the relationship between different Service types.

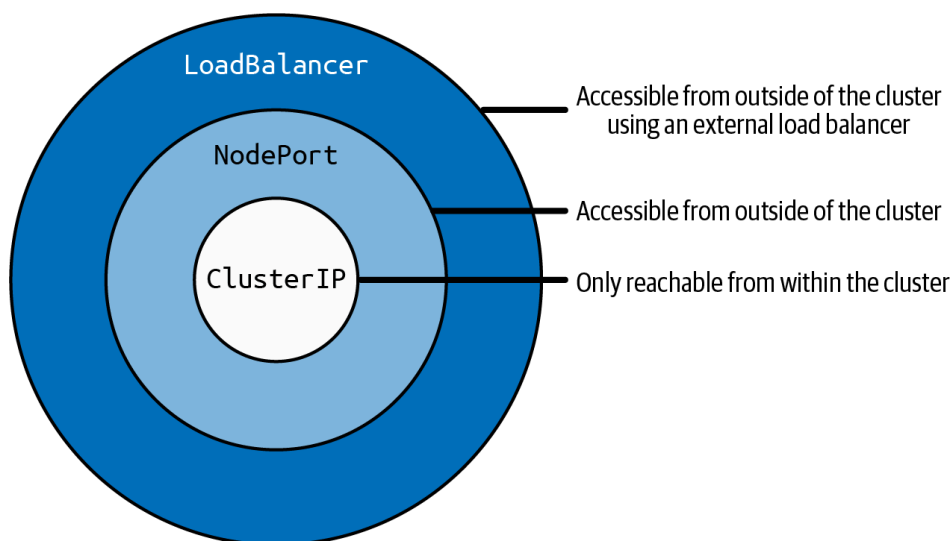


Figure 17-2. Network accessibility characteristics for Service types

For example, creating a Service of type `NodePort` means that the Service will bear the network accessibility characteristics of a `ClusterIP` Service type as well. In turn, a `NodePort` Service is accessible from within and from outside of the cluster. This chapter demonstrates each Service type by example. You will find references to the inherited exposure behavior in the following sections.

When to use which Service type?

When building a microservices architecture, the question arises of which Service type to choose to implement certain use cases. We briefly discuss this question here.

The `ClusterIP` Service type is suitable for use cases that call for exposing a microservice to other Pods within the cluster. Say you have a front-end microservice that needs to connect to one or many backend microservices. To properly implement the scenario, you'd stand up a `ClusterIP` Service that routes traffic to the backend Pods. The frontend Pods would then talk to that Service.

The `NodePort` Service type is often mentioned as a way to expose an application to consumers external to the cluster. Consumers will have to know the node's IP address and the statically assigned port to connect to the Service. That's problematic for multiple reasons. First, the node port is usually allocated dynamically. Therefore, you won't typically know it in advance. Second, providing the node's IP address will funnel the network traffic only through a single node so you will not have load balancing at your disposal. Finally, by opening a publicly available node port, you are at risk of increasing the attack surface of your cluster. For all these reasons, a `NodePort` Service is primarily used for development or testing purposes, and less so in production environments.

The `LoadBalancer` Service type makes the application available to outside consumers through an external IP address provided by an external load balancer. Network traffic will be distributed across multiple nodes in the cluster. This solution works great for production environments, but keep in mind that every provisioned load balancer will accrue costs and can lead to an expensive infrastructure bill. A more cost-effective solution is the use of an Ingress, discussed in [Chapter 18](#).

Port Mapping

A Service uses label selection to determine the set of Pods to forward traffic to. Successful routing of network traffic depends on the port mapping.

[Figure 17-3](#) shows a Service that accepts incoming traffic on port 80.

That's the port defined by the attribute `spec.ports[ ].port` in the manifest. Any incoming traffic is then routed toward the target port, represented by `spec.ports[ ].targetPort`.

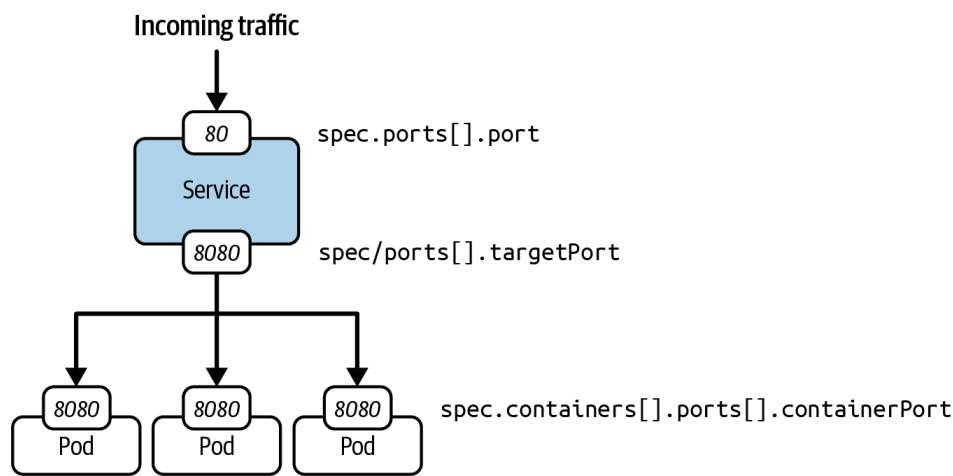


Figure 17-3. Service port mapping

The target port is the same port as defined by the container with `spec.containers[].ports[].containerPort` running inside the label-selected Pod. In this example, that's port 8080. The selected Pod(s) will receive traffic only if the Service's target port and the container port match.

Creating Services

You can create Services in a variety of ways, some of which are more appropriate for the exam as they provide a fast turnaround. Let's discuss the imperative approach first.

A Service needs to select a Pod with a matching label. The Pod created by the following `run` command is called `echoserver`, which exposes the application on the container port 8080. Internally, it automatically assigns the label key-value pair `run=echoserver` to the object:

```
$ kubectl run echoserver --image=k8s.gcr.io/echoserver:1.10 --restart=Never
--port=8080
pod/echoserver created
```

You can create a Service object using the `create service` command. Make sure to provide the Service type as a mandatory argument. Here we are using the type `clusterip`. The command-line option `--tcp` specifies the port mapping. Port 80 exposes the Service to incoming network traffic. Port 8080 targets the container port exposed by the Pod:

```
$ kubectl create service clusterip echoserver --tcp=80:8080
service/echoserver created
```

An even faster workflow of creating a Pod and Service together can be achieved with a `run` command and the `--expose` option. The following command creates both objects in one swoop while establishing the proper label selection. This command-line option is a good choice during the exam to save time if you are asked to create a Pod and a Service:

```
$ kubectl run echoserver --image=k8s.gcr.io/echoserver:1.10 --restart=Never
--port=8080 --expose
service/echoserver created
pod/echoserver created
```

It's actually more common to use a Deployment and Service that work together. The following set of commands creates a Deployment with five replicas and then uses the `expose deployment` command to instantiate the Service object. The port mapping can be provided with the options `--port` and `--target-port`:

```
$ kubectl create deployment echoserver --image=hashicorp/http-echo:1.0.0 \
--replicas=5
deployment.apps/echoserver created
$ kubectl expose deployment echoserver --port=80 --target-port=8080
service/echoserver exposed
```

[Example 17-1](#) shows the representation of a Service in the form of a YAML manifest. The Service declares the key-value `app=echoserver` for label selection and defines the port mapping from 80 to 8080.

Example 17-1. A Service defined by a YAML manifest

```
apiVersion: v1
kind: Service
metadata:
  name: echoserver
spec:
  selector:
    run: echoserver ❶
  ports: ❷
  - port: 80
    targetPort: 8080
```

- ❶ Selects all Pods with the given label assignment.
- ❷ Defines incoming and outgoing ports of the Service. The outgoing port needs to match the container port of the selected Pods.

The Service YAML manifest shown does not assign an explicit type. A Service object that does not specify a value for the attribute `spec.type` will default to `ClusterIP` upon creation.

Listing Services

Listing all Services presents a table view that includes the Service type, the ClusterIP address, an optional external IP address, and the incoming

port(s). Here, you can see the output for the `echoserver` Pod we created earlier:

```
$ kubectl get services
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
echoserver	ClusterIP	10.109.241.68	<none>	80/TCP	6s

Kubernetes assigns a ClusterIP address given that the Service type is `ClusterIP`. An external IP address is not available for this Service type. The Service is accessible on port 80.

Rendering Service Details

You may want to drill into the details of a Service for troubleshooting purposes. That might be the case if the incoming traffic to a Service isn't routed properly to the set of Pods you expect to handle the request.

The `describe service` command renders valuable information about the configuration of a Service. The configuration relevant to troubleshooting a Service is the value of the fields `Selector`, `IP`, `Port`, `TargetPort`, and `Endpoints`. A common source of misconfiguration is incorrect label selection and port assignment. Make sure that the selected labels are actually available in the Pods intended to route traffic to and that the target port of the Service matches the exposed container port of the Pods.

Take a look at the output of the following `describe` command. It's the details for a Service created for five Pods controlled by a Deployment. The `Endpoints` attribute lists a range of endpoints, one for each of the Pods:

```
$ kubectl describe service echoserver
```

```
Name: echoserver
Namespace: default
Labels: app=echoserver
Annotations: <none>
Selector: app=echoserver
Type: ClusterIP
IP Family Policy: SingleStack
IP Families: IPv4
IP: 10.109.241.68
IPs: 10.109.241.68
Port: <unset> 80/TCP
TargetPort: 8080/TCP
Endpoints: 172.17.0.4:8080,172.17.0.5:8080,172.17.0.7:8080 + 2 more.
Session Affinity: None
Events: <none>
```

An endpoint is a resolvable network endpoint, which serves as the virtual IP address and container port of a Pod. If a Service does not render any endpoints, then you are likely dealing with a misconfiguration. Use the EndpointSlice API to interact with the endpoints.

EndpointSlice is Kubernetes' scalable service discovery mechanism that maps Services to their backing Pod network endpoints. Instead of storing all endpoints in a single object like the deprecated Endpoints API, EndpointSlice distributes this information across multiple smaller objects, with each slice containing up to 100 endpoints by default. The following command lists the endpoints for the Service named `echoserver` with the assigned label `app=echoserver`:

```
$ kubectl get endpointslices -l app=echoserver
```

NAME	ADDRESSTYPE	PORTS	ENDPOINTS
echoserver-js2xj	IPv4	8080	10.244.0.10,10.244.0.11,10.244.0.9.

The details of the endpoints give away the full list of IP addresses and ports combinations:

```
$ kubectl describe endpointslice echoserver-js2xj
```

```
Name:          echoserver-js2xj
Namespace:     default
Labels:        app=echoserver
               endpointslice.kubernetes.io/managed-by=endpointslice-controller
               kubernetes.io/service-name=echoserver
Annotations:   endpoints.kubernetes.io/last-change-trigger-time: 2025-10-25T1
AddressType:   IPv4
Ports:
  Name      Port  Protocol
  ----      -
  <unset>   8080  TCP
Endpoints:
- Addresses: 10.244.0.10
  Conditions:
    Ready:    true
    Hostname: <unset>
    TargetRef: Pod/echoserver-85df578d68-q5r57
    NodeName: minikube
    Zone:     <unset>
    ...
```

The ClusterIP Service Type

ClusterIP is the default Service type. It exposes the Service on a cluster-internal IP address. That means the Service can be accessed only from a Pod running inside of the cluster and not from outside of the cluster

(e.g., if you were to make a call to the Service from your local machine).

[Figure 17-4](#) illustrates the accessibility of a Service with type `ClusterIP`.

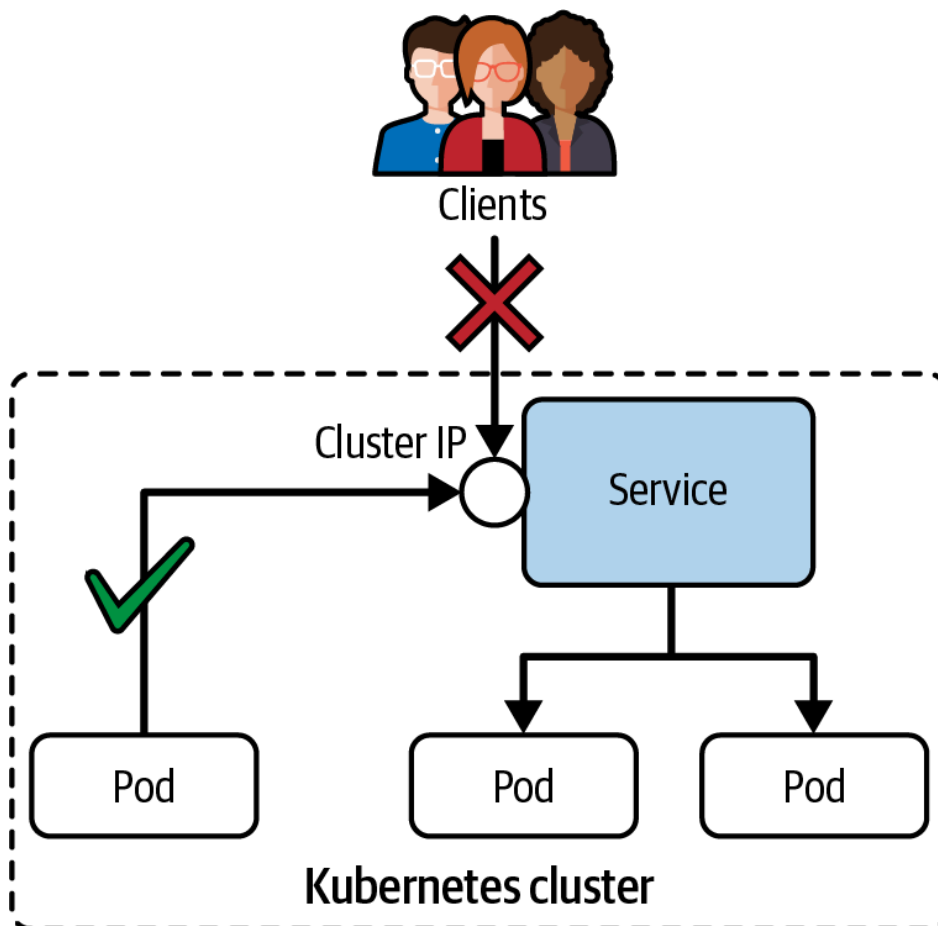


Figure 17-4. Accessibility of a Service with the type `ClusterIP`

Creating and Inspecting the Service

We will create a Pod and a corresponding Service to demonstrate the runtime behavior of the `ClusterIP` Service type. The Pod named `echoserver` exposes the container port 8080 and specifies the label `app=echoserver`. The Service defines port 5005 for incoming traffic, which is forwarded to outgoing port 8080. The label selection matches the Pod we set up:

```
$ kubectl run echoserver --image=k8s.gcr.io/echoserver:1.10 --restart=Never  
--port=8080 -l app=echoserver  
pod/echoserver created  
$ kubectl create service clusterip echoserver --tcp=5005:8080  
service/echoserver created
```

Inspecting the live object with the command `kubectl get service echoserver -o yaml` will render the assigned ClusterIP address.

[Example 17-2](#) shows an abbreviated version of the Service runtime representation.

Example 17-2. A ClusterIP Service object at runtime

```
apiVersion: v1
kind: Service
metadata:
  name: echoserver
spec:
  type: ClusterIP ❶
  clusterIP: 10.96.254.0 ❷
  selector:
    app: echoserver
  ports:
    - port: 5005
      targetPort: 8080
      protocol: TCP
```

❶ The Service type set to ClusterIP

❷ The ClusterIP address assigned to the Service at runtime

The ClusterIP address that makes the Service available in this example is 10.96.254.0 . Listing the Service object is an alternative way to render the information we need to make a call to the Service:

```
$ kubectl get service echoserver
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
echoserver	ClusterIP	10.96.254.0	<none>	5005/TCP	8s

Next up, we'll try to make a call to the Service.

Accessing the Service

You can access the Service using a combination of the ClusterIP address and the incoming port: 10.96.254.0:5005 . Making a request from any other machine residing outside of the cluster will fail, as illustrated by the following `wget` command:

```
$ wget 10.96.254.0:5005 --timeout=5 --tries=1
--2021-11-15 15:45:36-- http://10.96.254.0:5005/
Connecting to 10.96.254.0:5005... failed: Operation timed out.
Giving up.
```

Accessing the Service from a Pod from within the cluster properly routes the request to the Pod matching the label selection:

```
$ kubectl run tmp --image=busybox:1.36.1 --restart=Never -it --rm \
  -- wget 10.96.254.0:5005
Connecting to 10.96.254.0:5005 (10.96.254.0:5005)
```

```

saving to 'index.html'
index.html          100% |*****|          408    0:00:00
'index.html' saved
pod "tmp" deleted

```

Apart from using the ClusterIP address and the port, you can also discover a Service by DNS name and environment variables available to containers.

Discovering the Service by DNS lookup

Kubernetes registers every Service by its name with the help of its DNS service named [CoreDNS](#). Internally, CoreDNS will store the Service name as a hostname and maps it to the ClusterIP address. Accessing a Service by its DNS name instead of an IP address is much more convenient and expressive when building microservice architectures because IP addresses are ephemeral and unpredictable, whereas the labels are declarative and known.

You can verify the correct service discovery by running a Pod in the same namespace that makes a call to the Service by using its hostname and incoming port:

```

$ kubectl run tmp --image=busybox:1.36.1 --restart=Never -it --rm \
  -- wget echoserver:5005
Connecting to echoserver:5005 (10.96.254.0:5005)
saving to 'index.html'
index.html          100% |*****|          408    0:00:00
'index.html' saved
pod "tmp" deleted

```

It's not uncommon to make a call from a Pod to a Service that lives in a different namespace. Referencing just the hostname of the Service does not work across namespaces. You need to append the namespace as well. The following makes a call from a Pod in the `other` namespace to the Service in the `default` namespace:

```

$ kubectl run tmp --image=busybox:1.36.1 --restart=Never -it --rm \
  -n other -- wget echoserver.default:5005
Connecting to echoserver.default:5005 (10.96.254.0:5005)
saving to 'index.html'
index.html          100% |*****|          408    0:00:00
'index.html' saved
pod "tmp" deleted

```

The full hostname for a Service is `echoserver.default.svc.cluster.local`. The string `svc` describes the type of resource we are communicating with. CoreDNS uses the de-

fault value `cluster.local` as a domain name (which is configurable if you want to change it). You do not have to spell out the full hostname when communicating with a Service.

Discovering the Service by environment variables

You may find it easier to use the Service connection information directly from the application running in a Pod. The kubelet makes the ClusterIP address and port for every active Service available as environment variables. The naming convention for Service-related environment variables are `<SERVICE_NAME>_SERVICE_HOST` and `<SERVICE_NAME>_SERVICE_PORT`.

AVAILABILITY OF SERVICE ENVIRONMENT VARIABLES

Make sure you create the Service before instantiating the Pod. Otherwise, those environment variables won't be populated. This is why most developers avoid relying on these environment variables, as they rely on the creation (or deletion) order of objects.

You can check on the actual key-value pairs by listing the environment variables of the container, as follows:

```
$ kubectl exec -it echoserver -- env
ECHOSERVER_SERVICE_HOST=10.96.254.0
ECHOSERVER_SERVICE_PORT=8080
...
```

The name of the Service, `echoserver`, does not include any special characters. That's why the conversion to the environment variable key is easy; the Service name was simply uppercase to conform to environment variable naming conventions. Any special characters (such as dashes) in the Service name will be replaced by underscore characters. You need to make sure that the Service has been created before starting a Pod if you want those environment variables populated.

The NodePort Service Type

Declaring a Service with type `NodePort` exposes access through the node's IP address and can be resolved from outside of the Kubernetes cluster. The node's IP address can be reached in combination with a port number in the range of 30000 and 32767 (also called the node port), assigned automatically upon the creation of the Service. [Figure 17-5](#) illustrates the routing of traffic to Pods via a `NodePort`-type Service.

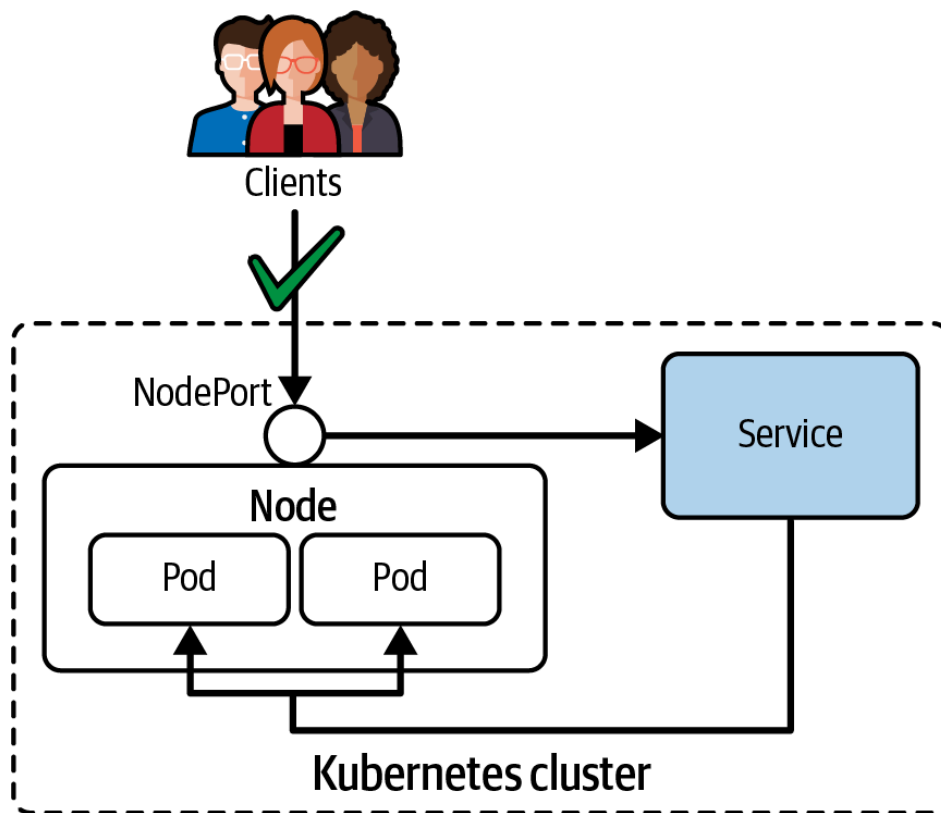


Figure 17-5. Accessibility of a Service with the type `NodePort`

The node port is opened on every node in the cluster, and its value is global and unique at the cluster-scope level. To avoid port conflicts, it's best to not define the exact node port and to let Kubernetes find an available port.

Creating and Inspecting the Service

The next two commands create a Pod and a Service of type `NodePort`. The only difference here is that `nodeport` is provided instead of `clusterip` as a command-line option:

```
$ kubectl run echoserver --image=k8s.gcr.io/echoserver:1.10 --restart=Never
--port=8080 -l app=echoserver
pod/echoserver created
$ kubectl create service nodeport echoserver --tcp=5005:8080
service/echoserver created
```

The runtime representation of the Service object is shown in [Example 17-3](#). It's important to point out that the node port will be assigned automatically. Keep in mind `NodePort` (capital *N*) is the Service type, whereas `nodePort` (lowercase *n*) is the key for the value.

Example 17-3. A `NodePort` Service object at runtime

```
apiVersion: v1
kind: Service
metadata:
  name: echoserver
```

```
spec:
  type: NodePort
  clusterIP: 10.96.254.0
  selector:
    app: echoserver
  ports:
  - port: 5005
    nodePort: 30158
    targetPort: 8080
    protocol: TCP
```

❶ The Service type set to `NodePort`

❷ The statically-assigned node port that makes the Service accessible from outside of the cluster

Once the Service is created, you can list it. You will find that the port representation contains the statically assigned port that makes the Service accessible:

```
$ kubectl get service echoserver
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
echoserver	NodePort	10.101.184.152	<none>	5005:30158/TCP	5s

In this output, the node port is 30158 (identifiable by the separating colon). The incoming port 5005 is still available for the purpose of resolving the Service from within the cluster.

Accessing the Service

From within the cluster, you can still access the Service using the ClusterIP address and port number. This Service displays exactly the same behavior as if it were of type `ClusterIP`:

```
$ kubectl run tmp --image=busybox:1.36.1 --restart=Never -it --rm \
  -- wget 10.101.184.152:5005
Connecting to 10.101.184.152:5005 (10.101.184.152:5005)
saving to 'index.html'
index.html      100% |*****|      414  0:00:00
'index.html' saved
pod "tmp" deleted
```

From outside of the cluster, you need to use the IP address of any worker node in the cluster and the statically assigned port. One way to determine the worker node's IP address is by rendering the node details. Another option is to use the `status.hostIP` attribute value of a Pod, which is the IP address of the worker node the Pod runs on.

The node IP address here is `192.168.64.15`. It can be used to call the Service from outside of the cluster:

```
$ kubectl get nodes -o \
  jsonpath='{ $.items[*].status.addresses[?(@.type=="InternalIP")].address }'
192.168.64.15
$ wget 192.168.64.15:30158
--2021-11-16 14:10:16-- http://192.168.64.15:30158/
Connecting to 192.168.64.15:30158... connected.
HTTP request sent, awaiting response... 200 OK
Length: unspecified [text/plain]
Saving to: 'index.html'
...
```

The LoadBalancer Service Type

The last Service type to discuss in this book is the `LoadBalancer`. This Service type provisions an external load balancer, primarily available to Kubernetes cloud providers, which exposes a single IP address to distribute incoming requests to the cluster nodes. The implementation of the load balancing strategy (e.g., round-robin) is up to the cloud provider.

LOAD BALANCERS FOR ON-PREMISES KUBERNETES CLUSTERS

Kubernetes does not offer a native load balancer solution for on-premises clusters. Cloud providers are in charge of providing an appropriate implementation. The [MetalLB project](#) aims to fill the gap.

[Figure 17-6](#) shows an architectural overview of the `LoadBalancer` Service type.

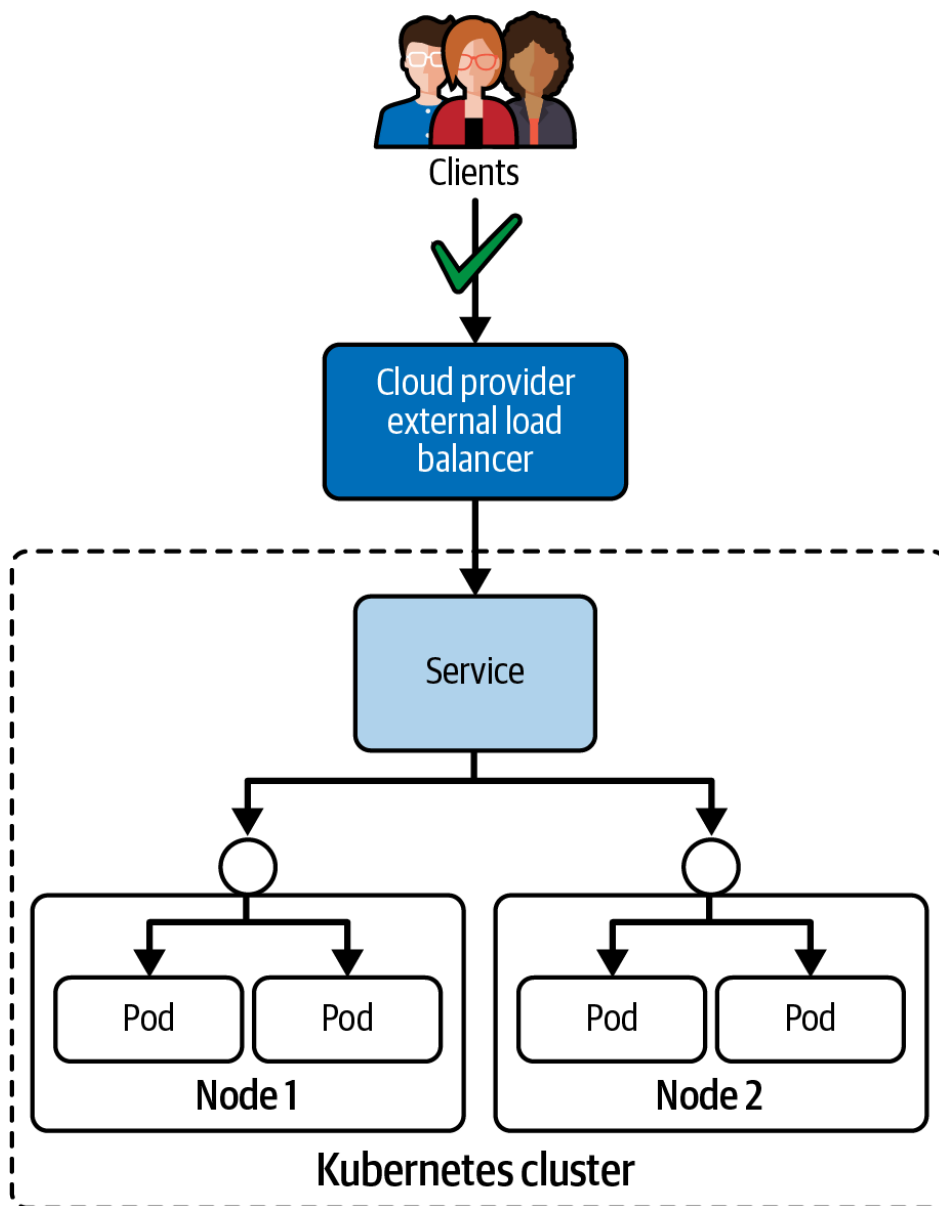


Figure 17-6. Accessibility of a Service with the type `LoadBalancer`

As you can see from the illustration, the load balancer routes traffic between different nodes, as long as the targeted Pods fulfill the requested label selection.

Creating and Inspecting the Service

To create a Service as a load balancer, set the type to `LoadBalancer` in the manifest or by using the `create service loadbalancer` command:

```
$ kubectl run echoserver --image=k8s.gcr.io/echoserver:1.10 --restart=Never
  --port=8080 -l app=echoserver
pod/echoserver created
$ kubectl create service loadbalancer echoserver --tcp=5005:8080
service/echoserver created
```

The runtime characteristics of a `LoadBalancer` Service type look similar to the ones provided by the `NodePort` Service type. The main difference

is that the external IP address column has a value, as shown in [Example 17-4](#).

Example 17-4. A LoadBalancer Service object at runtime

```
apiVersion: v1
kind: Service
metadata:
  name: echoserver
spec:
  type: LoadBalancer ❶
  clusterIP: 10.96.254.0
  loadBalancerIP: 10.109.76.157 ❷
  selector:
    app: echoserver
  ports:
    - port: 5005
      targetPort: 8080
      nodePort: 30158
      protocol: TCP
```

❶ The Service type set to LoadBalancer

❷ The external IP address assigned to the Service at runtime

Listing the Service renders the external IP address, which is 10.109.76.157 , as demonstrated by this command:

```
$ kubectl get service echoserver
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
echoserver	LoadBalancer	10.109.76.157	10.109.76.157	5005:30642/TCP

Given that the external load balancer needs to be provisioned by the cloud provider, it may take a little time until the external IP address becomes available.

Accessing the Service

To call the Service from outside of the cluster, use the external IP address and its incoming port:

```
$ wget 10.109.76.157:5005
--2021-11-17 11:30:44-- http://10.109.76.157:5005/
Connecting to 10.109.76.157:5005... connected.
HTTP request sent, awaiting response... 200 OK
Length: unspecified [text/plain]
Saving to: 'index.html'
...
```

As discussed, a `LoadBalancer` Service is also accessible in the same way as you would access a `ClusterIP` or `NodePort` Service.

Summary

Kubernetes assigns a unique IP address for every Pod in the cluster. Pods can communicate with each other using that IP address; however, you cannot rely on the IP address to be stable over time. That's why Kubernetes provides the Service resource type.

A Service forwards network traffic to a set of Pods based on label selection and port mappings. Every Service needs to assign a type that determines how the Service becomes accessible from within or outside of the cluster. The Service types relevant to the exam are `ClusterIP`, `NodePort`, and `LoadBalancer`. CoreDNS, the DNS server for Kubernetes, allows Pods to access the Service by hostname from the same and other namespaces.

Exam Essentials

Understand the purpose of a Service

Pod-to-Pod communication via their IP addresses doesn't guarantee a stable network interface over time. A restart of the Pod will lease a new virtual IP address. The purpose of a Service is to provide that stable network interface so that you can operate complex microservice architecture that runs in a Kubernetes cluster. In most cases, Pods call a Service by hostname. The hostname is provided by the DNS server named CoreDNS running as a Pod in the `kube-system` namespace.

Practice how to access a Service for each type

The exam expects you to understand the differences between the Service types `ClusterIP`, `NodePort`, and `LoadBalancer`. Depending on the assigned type, a Service becomes accessible from inside the cluster or from outside the cluster.

Work through Service troubleshooting scenarios

It's easy to get the configuration of a Service wrong. Any misconfiguration won't allow network traffic to reach the set of Pods it was intended for. Common misconfigurations include incorrect label selection and port assignments. The `kubectl get endpoints` command will give you an idea of which Pods a Service can route traffic to.

Sample Exercises

Solutions to these exercises are available in [Appendix A](#).

1. You need to expose a web application that runs on port 80 and has a metrics endpoint on port 9090. The application should be accessible from outside the cluster.

Create a Deployment named `webapp` using the image

`nginxdemos/hello:0.4-plain-text` with three replicas.

Create a Service named `webapp-service` of type `NodePort` that exposes port 80 as port 80 (name it `web`), and exposes port 9090 as port 9090 (name it `metrics`). Set the node port for the `web` port to 30080.

Use appropriate selectors to target your Deployment.

Verify the Service is working by accessing it through the ClusterIP.

2. You have a backend database and a frontend application. The frontend needs to connect to the database using service discovery. Both should only be accessible within the cluster.

Create a Deployment named `database` with one replica using image `mysql:9.4.0` with these environment variables:

`MYSQL_ROOT_PASSWORD=secretpass` and `MYSQL_DATABASE=myapp`.

Create a ClusterIP Service named `database-service` that exposes port 3306 and targets the database Pods.

Create a Deployment named `frontend` with two replicas using image `busybox:1.35` that runs a command to continuously test the database connection using the following command: `sh -c "while true; do nc -zv database-service 3306; sleep 5; done"`

Verify that the `frontend` Pods can resolve and connect to the database Service.